99 Übungsaufgaben

99 Übungsaufgaben aus den Folien

00 Übersicht

Lösungen sind eingeklappt (Pfeil anklicken). Wo die Folie keine Lösung zeigt, sind es meine eigenen** Lösungen, bitte gegenprüfen. Lösungen von Folien (Code) stehen direkt in den Themen-Notizen.**

Vorlesung 2 (12./13.10.)

Variablendeklaration

Welche Deklarationen sind syntaktisch nicht korrekt (Compiler-Fehler)? (02 Variablen)

```
int Boundary1;       integer boundary2;   float 1st;
String first name;   double int;          String course01;
int class;           int size;            int accountNr;
int java;            double accountbalance;
char Char;           String string;       string String;   long Long;
```

Lösung
Fehler:
- `integer boundary2;` – `integer` ist kein Typ (richtig: `int`)
- `float 1st;` – Name beginnt mit Ziffer
- `String first name;` – Leerzeichen im Namen
- `double int;` – `int` ist reserviertes Wort
- `int class;` – `class` ist reserviertes Wort
- `string String;` – `string` (klein) ist kein Typ, `String` ist groß

Korrekt, auch wenn teils unschön: `Boundary1`, `course01`, `size`, `accountNr`, `java` (kein reserviertes Wort), `accountbalance`, `char Char`, `String string`, `long Long` (Java unterscheidet Groß/Klein).

Namenskonventionen

Ok oder Verbesserungsvorschlag? (02 Variablen)

Beschreibung	Deklaration
Obergrenze bei Gauss	`int n;`
Postleitzahl	`String plz;`
Kontostand	`double accountbalance;`
Kursname	`String course;`

Beschreibung	Deklaration
Kontonummer	`int account;`
Seitenlänge Quadrat	`double a;`
Größe/Volumen eines Pakets	`double value;`
x-Koordinate	`double x;`
Zustand ready	`boolean is_ready;`

Lösung (nach Folie, sinngemäß)
- `n` → `boundary` (sprechend)
- `plz` → `postCode` (englisch), `plz` ist „etwas unschön, da deutsch“, darf aber bleiben
- `accountbalance` → `accountBalance` (camelCase), oder `balance`
- `course` → `courseName`
- `account` → `accountNumber` bzw. `accountNr`
- `a` → `length`
- `value` → `volume`
- `x` → `xCoordinate` oder lassen
- `is_ready` → `isReady` (kein Unterstrich)

Zweierkomplement

1. Tabelle für n = 4 vervollständigen: siehe 02b Zweierkomplement.
2. Was ergibt `2147483647 + 1`? Logisch lösen.

Lösung
`2147483647` ist `Integer.MAX_VALUE` = `0111…1`. Plus 1 ergibt `1000…0`, im Zweierkomplement der **kleinste** Wert: **−2.147.483.648** (Überlauf, kein Fehler).

Implizite Typkonversion

Wo tritt kein Informationsverlust auf (Anweisung korrekt)? (02a Numerische Datentypen)
1. `double price = 3;`
2. `int max = 3.0;`
3. `float value = 3;`
4. `int degree = 3f;`
5. `int number = 3l;`
6. `long seconds = 3;`

Lösung (nach Wertebereich)
1, 3, 6 (`int` → `double`/`float`/`long`: Zieltyp hat größeren Wertebereich).
2, 4, 5 (`double`/`float`/`long` → `int`: Informationsverlust möglich → Compilerfehler).

char

`int sum = u + v + w + x;` mit `int u = 49; char v = '4'; int w = '4'; char x = 49;` (02c char und Zeichensätze)
> **Lösung**
> `'4'` = 52 (da `'5'` = 53). `x` = Zeichen `'1'`, als Zahl 49. 49 + 52 + 52 + 49 = **202**

Übung 1 Tapete

20 m Tapete, Wand 3 m hoch: wie viele Bahnen, wie viel Verschnitt? (02e Arithmetische Operatoren)
> **Lösung**

> `20 / 3` = **6 Bahnen**, `20 % 3` = **2 m Verschnitt**.

Übung 2 Sekunden

Wie vielen Minuten entsprechen 35000 Sekunden, wie viele Sekunden bleiben übrig?
> **Lösung**
> `35000 / 60` = **583 Minuten**, `35000 % 60` = **20 Sekunden**.

Vorlesung 3 (19.10.)

Bedingungen formulieren

Mit `int i, j, k` und `boolean a, b, c`: (03a Operatoren in Bedingungen)

Text	Java
i, j und k sind alle verschieden von 0	?
i ist durch 17 teilbar und echt positiv	?
j ist ungerade und liegt echt zwischen 20 und 40	?
k ist 0 und es gilt nicht a	?
es gilt genau a oder genau b	?
es gilt nicht c	?
es gilt b oder nicht c, und immer gilt i ist echt kleiner 0	?

Lösung (eigene)

```
i != 0 && j != 0 && k != 0
i % 17 == 0 && i > 0
j % 2 != 0 && j > 20 && j < 40
k == 0 && !a
a ^ b
!c
(b || !c) && i < 0
```

Bedingungen prüfen

Korrekt oder nicht? Mit `int i, j, k` und `boolean a, b`:
1. `i && (j || k)`
2. `i % 32 || j > 23 * 5 && !b`
3. `i*j*k=0`
4. `i % 4 == 0 && i % 100 > 0`
5. `i < j && a && b`
6. `a * b == 0`
7. `!(a > b && i < j)`

Lösung (eigene, mehrere Korrekturen möglich)
1. `&&`/`||` brauchen `boolean`, nicht `int` → `i != 0 && (j != 0 || k != 0)`
2. `i % 32` ist `int` → `i % 32 != 0 || j > 23 * 5 && !b`
3. `=` ist Zuweisung, nicht Vergleich → `i * j * k == 0`
4.
5.
6. Arithmetik mit `boolean` nicht erlaubt → `!(a && b)`

7. relationale Operatoren (`>`) brauchen numerische Operanden → z.B. `!(a && !b && i < j)`

Vorlesung 4 (22.10.)

Prä- und Postfix-Ausgaben

Wie sieht jeweils die Ausgabe aus? (04a Schleifen)

```java
// Beispiel 1
int i = 2;
while (i >= 0) {
    System.out.print(--i);
}
```

```java
// Beispiel 2
int i = 2;
while (i >= 0) {
    System.out.print(i--);
}
```

```java
// Beispiel 3
int i = 2;
do {
    System.out.print(i);
} while (--i >= 0);
```

```java
// Beispiel 4
int i = 2;
do {
    System.out.print(i);
} while (i-- >= 0);
```

Lösung (eigene Herleitung)

1. `10-1` (`--i` ändert zuerst: gibt 1, 0, −1 aus; bei i = −1 endet die Schleife)
2. `210` (`i--` liefert zuerst den alten Wert: 2, 1, 0)
3. `210` (ausgeben, dann `--i >= 0` prüfen: 2, 1, 0, dann i = −1 → Ende)
4. `210-1` (`i-- >= 0` vergleicht den **alten** Wert, daher läuft die Schleife noch einmal mit i = −1)

Weitere Aufgaben mit Lösung in den Themen-Notizen: Schaltjahr (04 if-Anweisung), `while`/`do-while`/`for` (04a Schleifen), Tage pro Monat (04b Switch).